Can the clients notice the failures? (either during order requests or the final order checking phase) or are they transparent to the clients?

Clients may briefly notice failures **only if** a request is made during the window when the current leader has crashed **and** leader re-election hasn't completed yet. During that small window, requests routed to the dead leader will fail

Once leader re-election is completed, the frontend routes requests to the new leader transparently. From this point onward, failures are mostly hidden from the client, achieving near-transparency.

The frontend service monitors leader health (/health) and performs automatic re-election using the highest ID among responsive replicas. After leader re-election, requests are transparently routed to the new leader by the frontend. Therefore failures are mostly transparent to the clients except for very short failover windows.

Do all the order service replicas end up with the same database file?

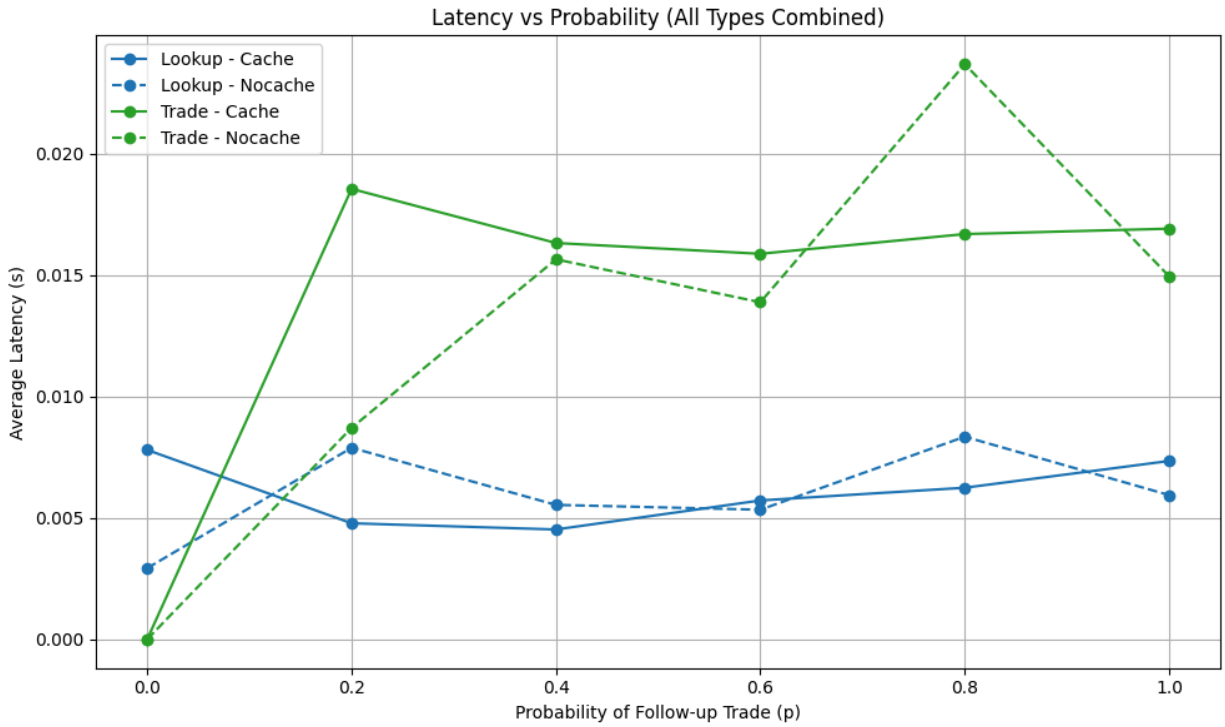
Yes, eventually all the replicas end up with the same orders.csv file. System uses two flows to ensure this:

- /propagate endpoint:
  When a new order is created, the leader sends the order to all followers using a POST request to /propagate. Followers append this to their local in-memory state.

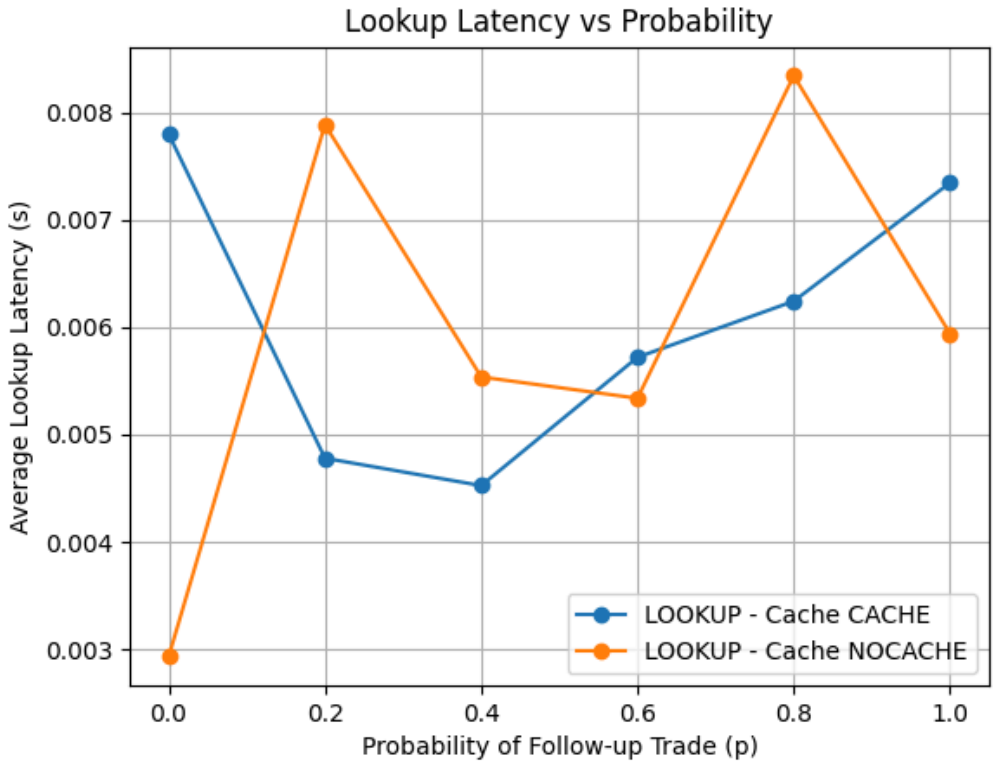
- /sync_missing?from_id=<x> endpoint:
  When a replica recovers from a crash, it runs the sync_with_peers() function, which contacts other replicas to fetch missing transactions since its last known transaction ID. These updates are applied to the local state.

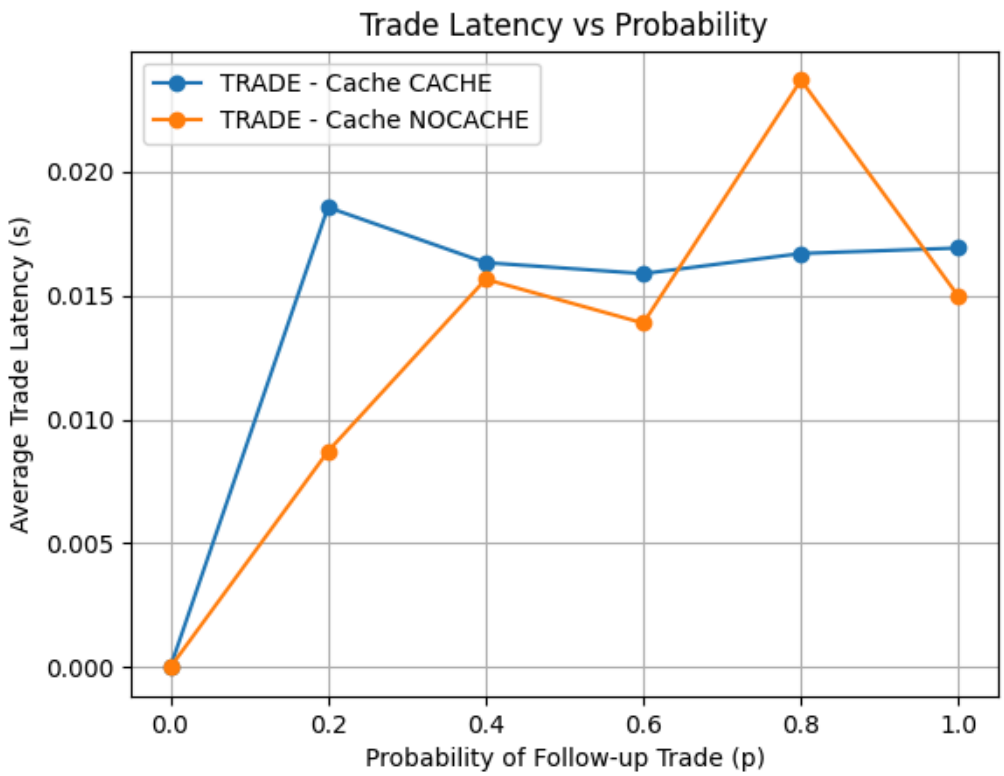
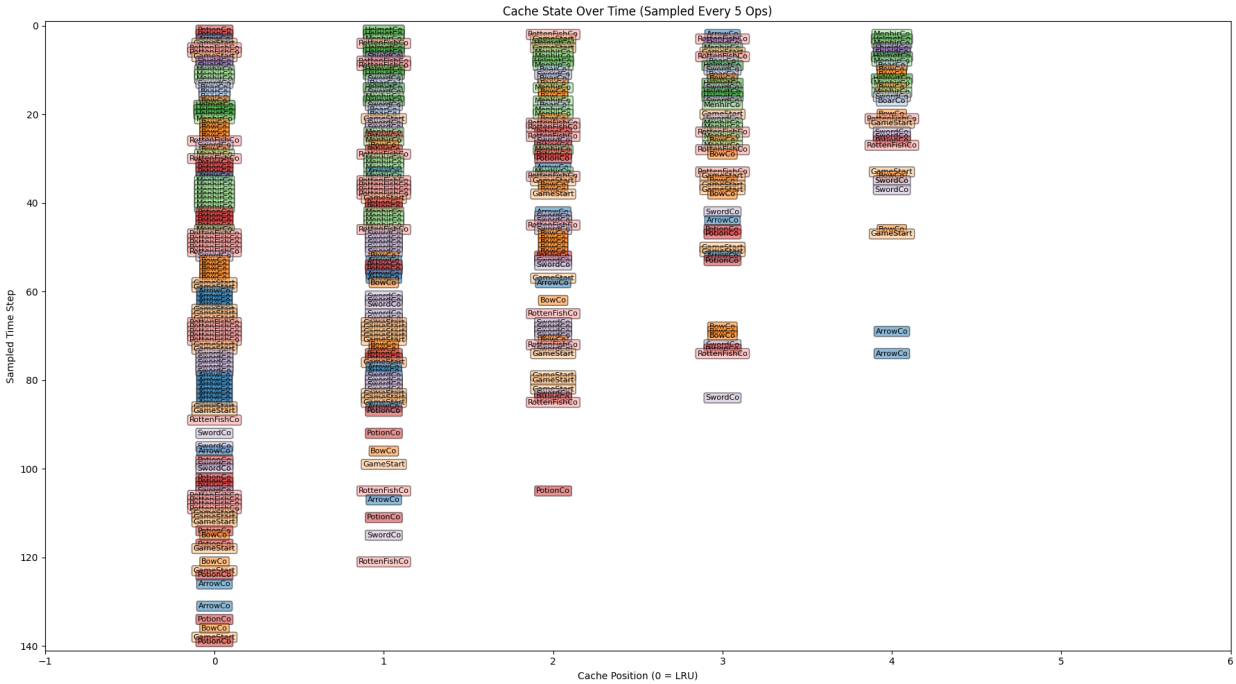
This will ensure that even if a replica is down when an order is placed, it will eventually catch up and replicate all missed data, resulting in identical orders_data and persisted files (orders.csv) across all replicas. replicas end up with the same database file, guaranteeing eventual consistency in the system.

Latency plots:



For trade requests, cache or no cache latencies would be similar as caching hasnt been implemented for trade requests.
For lookup requests, requests with cache have considerably lower latencies than requests without cache


Lookup Latency vs Probability
Average Lookup Latency (s)
0.008
0.007
0.006
0.005
0.004
0.003
0.0
0.2
0.4
0.6
0.8
1.0
Probability of Follow-up Trade (p)
LOOKUP - Cache CACHE
LOOKUP - Cache NOCACHE


Trade Latency vs Probability
Average Trade Latency (s)
0.020
0.015
0.010
0.005
0.000
0.0
0.2
0.4
0.6
0.8
1.0
Probability of Follow-up Trade (p)
TRADE - Cache CACHE
TRADE - Cache NOCACHE

Cache eviction plot: